

Consulta GET y POST

vamos a ejecutar una consulta con un formulario utilizando get y utilizando post para ver la respuesta... hagamos un ejemplo simple consultando a google via html y js

Es un buen ejemplo para explicar la diferencia entre **GET** y **POST**, pero hay una particularidad importante:

-  **Google sí acepta búsquedas mediante GET** (es el método que utiliza normalmente).
-  **Google no permite realizar búsquedas mediante POST** desde un formulario HTML común. Si intentás hacerlo, no obtendrás el comportamiento esperado porque el buscador está diseñado para recibir el parámetro `q` por la URL.

Por eso, para la práctica podemos hacer:

1. Un formulario **GET** que consulte Google (funciona).
2. Un formulario **POST** que envíe los datos a una página propia (`respuesta.html`) para observar cómo viajan los datos, o bien a un pequeño servidor Node.js/Java si quieren ver el procesamiento del lado del servidor.

Ejemplo 1 - Consulta con GET a Google

index.html

HTML

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Consulta GET</title>

  <style>
    body{
      font-family: Arial;
      width:500px;
      margin:40px auto;
    }

    input{
      width:300px;
      padding:8px;
    }
  </style>
</head>
<body>
  <input type="text" value="Buscar" />
  <input type="button" value="Buscar" />
</body>
</html>
```

Si el usuario escribe

Programación Orientada a Objetos

el navegador genera automáticamente la URL

https://www.google.com/search?q=Programación+Orientada+a+Objetos

Los datos viajan en la URL.

¿Cómo verlo desde JavaScript?

Podemos capturar el submit antes de enviarlo.

HTML

```
<script>

const formulario = document.querySelector("form");

formulario.addEventListener("submit", function(e){

    const texto = document.querySelector("input").value;

    console.log("Se enviará mediante GET");

    console.log(texto);

});

</script>
```

No cancelamos el envío, simplemente mostramos qué se enviará.

Ejemplo 2 - POST

Ahora cambia únicamente el método.

HTML

```
<form
  action="respuesta.html"
  method="POST">

  <input
    type="text"
```

```
name="nombre">

<button>
  Enviar
</button>

</form>
```

Aquí los datos **no aparecen en la URL**.

Se envían dentro del **cuerpo (Body)** de la petición HTTP.

¿Cómo ver el POST?

Abrir las herramientas del navegador:

F12

Luego

Network

Enviar el formulario.

Seleccionar la petición.

Veremos algo similar a:

Request Method

POST

y en

Form Data

nombre=Juan

Comparación visual

GET

POST

Los datos viajan en la
URL

Los datos viajan en el
Body

GET	POST
Puede guardarse como favorito	No
Tiene límite práctico de longitud	Puede enviar mucha más información
Visible en el historial	No aparece en la URL
Ideal para búsquedas	Ideal para altas, modificaciones y login

¿Y JavaScript puede hacer exactamente lo mismo?

Sí.

Con `fetch()` podemos enviar un GET:

JavaScript

```
const texto = "Java";

fetch("https://www.google.com/search?q=" + texto);
```

Aunque por las políticas de seguridad del navegador (CORS), no podremos leer la respuesta de Google desde JavaScript.

Y un POST sería:

JavaScript

```
fetch("/servidor", {

  method: "POST",

  headers: {
    "Content-Type": "application/json"
  },

  body: JSON.stringify({

    nombre: "Juan"

  })

});
```

Aquí el objeto JSON viaja en el **Body** de la petición.

Propuesta didáctica

Como están trabajando con **HTML, JavaScript y luego Servlets/JSP**, una práctica muy ilustrativa sería construir una pequeña aplicación con:

- **Página 1:** formulario `GET` que consulte Google.
- **Página 2:** formulario `POST` que envíe datos a un **Servlet Java**.
- **El Servlet** leerá los parámetros con `request.getParameter(...)` y mostrará una página de respuesta.

De esa forma los alumnos verán el recorrido completo de una petición HTTP tanto con **GET** como con **POST**, desde el navegador hasta el servidor Java.



Formulario HTML con POST y Servlet Java

Vamos a crear una aplicación sencilla que:

1. Muestre un formulario HTML.
2. Solicite nombre y apellido.
3. Envíe los datos mediante `POST`.
4. Un Servlet Java reciba los datos.
5. El Servlet genere una página HTML con la respuesta.

Usaremos:

- Java 17
- Maven
- Apache Tomcat 10.1
- Jakarta Servlet
- Visual Studio Code

En Tomcat 10 se utilizan los paquetes `jakarta.servlet` en lugar de los antiguos `javax.servlet`. Un Servlet que atiende solicitudes POST debe sobrescribir el método `doPost()`.  Jakarta EE +1

1. Crear el proyecto Maven

Abrimos una terminal en Visual Studio Code y ejecutamos:

```
</> Bash
```

```
mvn archetype:generate ^  
-DgroupId=com.universitas ^  
-DartifactId=formulario-post ^  
-DarchetypeArtifactId=maven-archetype-webapp ^  
-DinteractiveMode=false
```

En PowerShell o Linux/macOS se puede escribir en una sola línea:

</> Bash

```
mvn archetype:generate -DgroupId=com.universitas -DartifactId=formulario-post -DarchetypeArt:
```

Ingresamos al proyecto:

</> Bash

```
cd formulario-post
```

Abrimos la carpeta en Visual Studio Code:

</> Bash

```
code .
```

2. Estructura del proyecto

La estructura final será:

```
formulario-post  
|  
├── pom.xml  
|  
└── src  
    ├── main  
    │   ├── java  
    │   │   ├── com  
    │   │   │   ├── universidades  
    │   │   │   │   ├── servlet  
    │   │   │   │   │   ├── SaludoServlet.java  
    │   │   │   │   └──  
    │   │   └──  
    │   └── webapp  
    │       ├── index.html  
    │       ├── WEB-INF  
    │       │   └── web.xml
```

El arquetipo puede no crear la carpeta `src/main/java`, por lo que tendremos que crearla manualmente.

3. Configurar el archivo pom.xml

Reemplazamos completamente el contenido del pom.xml por lo siguiente:

</> XML

```
<?xml version="1.0" encoding="UTF-8"?>

<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="
    http://maven.apache.org/POM/4.0.0
    https://maven.apache.org/xsd/maven-4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>

  <groupId>com.universitas</groupId>
  <artifactId>formulario-post</artifactId>
  <version>1.0-SNAPSHOT</version>

  <!-- La aplicación se empaqueta como archivo WAR -->
  <packaging>war</packaging>

  <properties>
    <maven.compiler.source>17</maven.compiler.source>
    <maven.compiler.target>17</maven.compiler.target>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>

  <dependencies>

    <!-- API de Jakarta Servlet utilizada por Tomcat 10 -->
    <dependency>
      <groupId>jakarta.servlet</groupId>
      <artifactId>jakarta.servlet-api</artifactId>
      <version>6.0.0</version>
      <scope>provided</scope>
    </dependency>

  </dependencies>

  <build>

    <!-- Nombre del archivo WAR generado -->
    <finalName>formulario-post</finalName>

    <plugins>

      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-compiler-plugin</artifactId>
        <version>3.13.0</version>
```

```
<configuration>
  <release>17</release>
</configuration>
</plugin>

<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-war-plugin</artifactId>
  <version>3.5.1</version>

  <configuration>
    <failOnMissingWebXml>>false</failOnMissingWebXml>
  </configuration>
</plugin>

</plugins>

</build>

</project>
```

El empaquetado `war` hace que Maven genere un archivo desplegable en un servidor web como Tomcat. El Maven WAR Plugin reúne las clases, dependencias y recursos web de la aplicación dentro de ese archivo.  Apache Maven +1

La dependencia tiene:

```
</> XML
<scope>provided</scope>
```

Esto significa que Maven utiliza la API para compilar, pero no la agrega dentro del WAR porque Tomcat ya proporciona esas clases.

4. Crear el formulario HTML

Abrimos:

```
src/main/webapp/index.html
```

Reemplazamos su contenido por:

```
HTML
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
```



```
<meta
  name="viewport"
  content="width=device-width, initial-scale=1.0"
>

<title>Formulario POST</title>

<style>
  * {
    box-sizing: border-box;
  }
```

La parte más importante es:

HTML

```
<form action="saludar" method="POST">
```

Esto indica:

```
action="saludar"
```

La solicitud se enviará a la URL relativa:

```
/saludar
```

Y:

```
method="POST"
```

Los datos se enviarán mediante el método HTTP `POST`.

5. Importancia del atributo `name`

Los campos tienen:

HTML

```
<input name="nombre">
```

y:

HTML

```
<input name="apellido">
```

El atributo `name` determina el nombre del parámetro que recibirá el Servlet.

Conceptualmente, el cuerpo de la petición contendrá:

```
nombre=Juan&apellido=Perez
```

El atributo `id`, en cambio, se utiliza principalmente para relacionar el campo con su etiqueta o para seleccionarlo desde CSS y JavaScript.

6. Crear el Servlet

Creamos las carpetas:

```
src/main/java/com/universitas/servlet
```

Dentro creamos:

```
SaludoServlet.java
```

Código completo:

```
</> Java

package com.universitas.servlet;

import java.io.IOException;
import java.io.PrintWriter;

import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

/**
 * Servlet encargado de recibir Los datos enviados por
 * el formulario mediante una petición HTTP POST.
 */
@WebServlet("/saludar")
public class SaludoServlet extends HttpServlet {

    private static final long serialVersionUID = 1L;

    /**
     * Atiende las solicitudes HTTP POST realizadas
     * a la dirección /saludar.
     *
     * @param request contiene los datos enviados por el navegador
     * @param response permite construir la respuesta HTTP
     */
}
```

```
@Override
protected void doPost(
    HttpServletRequest request,
    HttpServletResponse response
) throws ServletException, IOException {

    /*
     * Indicamos que Los datos recibidos utilizan UTF-8.
     * Debe ejecutarse antes de Leer Los parámetros.
     */
    request.setCharacterEncoding("UTF-8");

    /*
     * Recuperamos Los parámetros enviados por el formulario.
     * Los nombres deben coincidir con el atributo name
     * de cada input.
     */
    String nombre = request.getParameter("nombre");
    String apellido = request.getParameter("apellido");

    /*
     * Configuramos el tipo de contenido de la respuesta.
     */
    response.setContentType("text/html;charset=UTF-8");

    /*
     * Obtenemos un escritor para generar la respuesta HTML.
     */
    PrintWriter out = response.getWriter();

    out.println("""
        <!DOCTYPE html>
        <html lang="es">

        <head>
            <meta charset="UTF-8">

            <meta
                name="viewport"
                content="width=device-width, initial-scale=1.0"
            >

            <title>Respuesta del servidor</title>

            <style>
                body {
                    margin: 0;
                    min-height: 100vh;
                    display: flex;
                    justify-content: center;
                    align-items: center;
                    font-family: Arial, sans-serif;
                    background-color: #f1f3f5;
                }
            </style>
        """);
}
```

```

    }

    .respuesta {
        width: 450px;
        padding: 30px;
        text-align: center;
        background-color: white;
        border-radius: 10px;
        box-shadow: 0 4px 15px rgba(0, 0, 0, 0.15);
    }

    h1 {
        color: #1565c0;
    }

    a {
        display: inline-block;
        margin-top: 20px;
        padding: 10px 18px;
        color: white;
        text-decoration: none;
        background-color: #1565c0;
        border-radius: 5px;
    }

    a:hover {
        background-color: #0d47a1;
    }
</style>
</head>

<body>

    <main class="respuesta">
        """);

out.println("<h1>Datos recibidos correctamente</h1>");

out.println(
    "<p>Hola, <strong>"
    + escaparHtml(nombre)
    + " "
    + escaparHtml(apellido)
    + "</strong>.</p>"
);

out.println(
    "<p>Esta respuesta fue generada por un Servlet Java.</p>"
);

out.println("""
    <a href="index.html">
        Volver al formulario

```

```

        </a>
    </main>

</body>

</html>
""");
}

/**
 * Reemplaza caracteres especiales para evitar que el contenido
 * ingresado por el usuario sea interpretado como código HTML.
 *
 * @param texto texto ingresado por el usuario
 * @return texto seguro para mostrar dentro de una página HTML
 */
private String escaparHtml(String texto) {

    if (texto == null) {
        return "";
    }

    return texto
        .replace("&", "&amp;")
        .replace("<", "&lt;")
        .replace(">", "&gt;")
        .replace("\"", "&quot;")
        .replace("'", "&#39;");
}
}

```

La anotación:

 Java

```
@WebServlet("/saludar")
```

relaciona la URL `/saludar` con la clase `SaludoServlet`.

El formulario apunta precisamente a esa dirección:

 HTML

```
<form action="saludar" method="POST">
```

7. ¿Qué hace `doPost()`?

Cuando el navegador envía el formulario mediante POST, Tomcat llama automáticamente a:

</> Java

```
protected void doPost(  
    HttpServletRequest request,  
    HttpServletResponse response  
)
```

Los parámetros cumplen funciones diferentes:

</> Java

```
HttpServletRequest request
```

Representa la solicitud enviada por el cliente. Permite leer:

- Parámetros del formulario.
- Encabezados HTTP.
- Cookies.
- Datos de sesión.
- Método HTTP utilizado.

</> Java

```
HttpServletResponse response
```

Representa la respuesta que el servidor enviará al navegador. Permite definir:

- Tipo de contenido.
- Código de estado.
- Encabezados.
- Cookies.
- Contenido HTML o JSON.

La documentación oficial de Jakarta establece que `HttpServlet` se extiende y se sobrescribe `doPost()` para atender solicitudes POST.  Jakarta EE +1

8. Leer los datos del formulario

Los valores se leen con:

</> Java

```
String nombre = request.getParameter("nombre");  
String apellido = request.getParameter("apellido");
```

Los nombres coinciden con los `name` del HTML:

</> HTML

```
<input name="nombre">
```

HTML

```
<input name="apellido">
```

La relación es:

HTML**Servlet**

```
name="nombre"
```

```
getParameter("nombre")
)
```

```
name="apellido"
```

```
getParameter("apellido")
)
```

Si escribimos:

Nombre: Juan

Apellido: Pérez

el Servlet recibirá:

Java

```
nombre = "Juan";
apellido = "Pérez";
```

9. Configurar **web.xml**

Abrimos:

```
src/main/webapp/WEB-INF/web.xml
```

Colocamos:

XML

```
<?xml version="1.0" encoding="UTF-8"?>

<web-app
  xmlns="https://jakarta.ee/xml/ns/jakartaee"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="
    https://jakarta.ee/xml/ns/jakartaee
    https://jakarta.ee/xml/ns/jakartaee/web-app_6_0.xsd"
  version="6.0">

  <display-name>Formulario POST con Java</display-name>
```

```
<welcome-file-list>
  <welcome-file>index.html</welcome-file>
</welcome-file-list>

</web-app>
```

No necesitamos declarar el Servlet en este archivo porque ya utilizamos:

```
<> Java
@WebServlet("/saludar")
```

El `web.xml` solo configura en este caso la página inicial.

10. Compilar el proyecto

Desde la raíz del proyecto ejecutamos:

```
<> Bash
mvn clean package
```

Maven:

1. Elimina compilaciones anteriores.
2. Compila la clase Java.
3. Copia los recursos web.
4. Genera el archivo WAR.

Si la compilación es correcta veremos:

```
BUILD SUCCESS
```

Dentro de la carpeta `target` aparecerá:

```
target/formulario-post.war
```

11. Instalar Tomcat

Descargamos y descomprimos Apache Tomcat 10.1, por ejemplo en:

```
C:\apache-tomcat-10.1
```

La estructura principal será:


```
apache-tomcat-10.1
```

```
|— bin  
|— conf  
|— lib  
|— logs  
|— webapps  
|— work
```

12. Desplegar la aplicación

Copiamos:

```
target/formulario-post.war
```

dentro de:

```
C:\apache-tomcat-10.1\webapps
```

Quedará:

```
C:\apache-tomcat-10.1\webapps\formulario-post.war
```

Tomcat utiliza el nombre del WAR como contexto de la aplicación.

Por eso la dirección será:

```
http://localhost:8080/formulario-post/
```

13. Iniciar Tomcat

Abrimos una terminal y vamos a:

```
</> Bash
```

```
cd C:\apache-tomcat-10.1\bin
```

Ejecutamos:

```
</> Bash
```

```
startup.bat
```

Para ver los mensajes del servidor directamente en la terminal también podemos ejecutar:

```
</> Bash
```

```
catalina.bat run
```

Esta segunda opción es útil durante el desarrollo porque permite ver inmediatamente cualquier error.

14. Abrir la aplicación

En el navegador ingresamos:

```
http://localhost:8080/formulario-post/
```

Se mostrará el formulario.

Ingresamos, por ejemplo:

```
Nombre: Ana
```

```
Apellido: López
```

Presionamos:

```
Enviar mediante POST
```

El navegador enviará una petición a:

```
http://localhost:8080/formulario-post/saludar
```

Y el Servlet responderá:

```
Datos recibidos correctamente
```

```
Hola, Ana López.
```

```
Esta respuesta fue generada por un Servlet Java.
```

15. Observar la petición POST

Para comprobar que el formulario realmente usa POST:

1. Abrimos la aplicación.
2. Presionamos **F12**.
3. Seleccionamos la pestaña **Network** o **Red**.

4. Enviamos el formulario.
5. Seleccionamos la petición llamada `saludar`.

Veremos:

```
Request Method: POST
```

En la sección correspondiente a los datos del formulario aparecerá:

```
nombre=Ana  
apellido=López
```

La URL será:

```
http://localhost:8080/formulario-post/saludar
```

Los datos no aparecen agregados a la URL porque se enviaron en el cuerpo de la petición.

16. Recorrido completo de la solicitud

El recorrido es:

```
Usuario  
↓  
Completa index.html  
↓  
<form method="POST" action="saludar">  
↓  
Tomcat recibe la petición  
↓  
@WebServlet("/saludar")  
↓  
SaludoServlet.doPost()  
↓  
request.getParameter(...)  
↓  
El Servlet construye una respuesta HTML  
↓  
El navegador muestra la respuesta
```

17. Diferencia respecto de GET

Con POST:

 HTML

```
<form action="saludar" method="POST">
```

La dirección queda:

```
http://localhost:8080/formulario-post/saludar
```

Los datos están en el cuerpo de la petición.

Con GET:

 HTML

```
<form action="saludar" method="GET">
```

La dirección tendría una forma similar a:

```
http://localhost:8080/formulario-post/saludar?nombre=Ana&apellido=Lopez
```

En ese caso, el Servlet debería implementar:

 Java

```
@Override
protected void doGet(
    HttpServletRequest request,
    HttpServletResponse response
) throws ServletException, IOException {

}
```

Resultado final

El ejemplo utiliza:

```
index.html
  ↓ POST
SaludoServlet
  ↓
Respuesta HTML
```

No utiliza JavaScript porque un formulario HTML puede realizar una petición POST directamente. JavaScript sería necesario más adelante para interceptar el envío y realizarlo con `fetch()` sin recargar completamente la página.

dame los pasos actualizados para educacionIT

Formulario HTML con POST y Servlet Java

Versión actualizada para EducaciónIT

Vamos a crear una aplicación web sencilla utilizando:

- Java 17
- Maven
- Apache Tomcat 10.1
- Jakarta Servlet
- Visual Studio Code

El formulario solicitará un nombre y un apellido, enviará los datos mediante **POST** y un Servlet Java devolverá una respuesta HTML.

1. Crear el proyecto Maven

Abrimos una terminal y ejecutamos:

En CMD de Windows

</> Bash

```
mvn archetype:generate ^  
-DgroupId=com.educacionIT ^  
-DartifactId=formulario-post ^  
-DarchetypeArtifactId=maven-archetype-webapp ^  
-DinteractiveMode=false
```

En PowerShell

</> PowerShell

```
mvn archetype:generate `   
-DgroupId=com.educacionIT `   
-DartifactId=formulario-post `   
-DarchetypeArtifactId=maven-archetype-webapp `   
-DinteractiveMode=false
```

También puede ejecutarse en una sola línea:

</> Bash

```
mvn archetype:generate -DgroupId=com.educacionIT -DartifactId=formulario-post -DarchetypeArt:
```

Ingresamos al proyecto:

```
</> Bash
```

```
cd formulario-post
```

Abrimos el proyecto en Visual Studio Code:

```
</> Bash
```

```
code .
```

2. Estructura final del proyecto

La estructura será:

```
formulario-post
├── pom.xml
├── src
│   ├── main
│   │   ├── java
│   │   │   ├── com
│   │   │   │   ├── educacionIT
│   │   │   │   │   ├── servlet
│   │   │   │   │   │   └── SaludoServlet.java
│   │   └── webapp
│   │       ├── index.html
│   │       ├── WEB-INF
│   │       └── web.xml
```

El arquetipo web de Maven puede no crear automáticamente la carpeta:

```
src/main/java
```

En ese caso, debemos crearla manualmente.

3. Configurar el archivo `pom.xml`

Reemplazamos el contenido completo de `pom.xml` por:

```
</> XML
```

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="
    http://maven.apache.org/POM/4.0.0
    https://maven.apache.org/xsd/maven-4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>

  <groupId>com.educacionIT</groupId>
  <artifactId>formulario-post</artifactId>
  <version>1.0-SNAPSHOT</version>

  <!-- La aplicación será empaquetada como archivo WAR -->
  <packaging>war</packaging>

  <properties>
    <maven.compiler.release>17</maven.compiler.release>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>

  <dependencies>

    <!-- API de Jakarta Servlet para Tomcat 10 -->
    <dependency>
      <groupId>jakarta.servlet</groupId>
      <artifactId>jakarta.servlet-api</artifactId>
      <version>6.0.0</version>
      <scope>provided</scope>
    </dependency>

  </dependencies>

  <build>

    <!-- Nombre del archivo WAR -->
    <finalName>formulario-post</finalName>

    <plugins>

      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-compiler-plugin</artifactId>
        <version>3.13.0</version>

        <configuration>
          <release>17</release>
        </configuration>
      </plugin>

      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-war-plugin</artifactId>
        <version>3.4.0</version>
```

```
        <configuration>
            <failOnMissingWebXml>false</failOnMissingWebXml>
        </configuration>
    </plugin>

</plugins>

</build>

</project>
```

El valor:

```
</> XML
<groupId>com.educacionIT</groupId>
```

identifica a la organización o grupo al cual pertenece el proyecto.

El valor:

```
</> XML
<packaging>war</packaging>
```

indica que Maven debe generar un archivo web desplegable en Tomcat.

La dependencia tiene:

```
</> XML
<scope>provided</scope>
```

Esto significa que la biblioteca Servlet se utiliza durante la compilación, pero no se incluye dentro del WAR porque Tomcat ya la proporciona.

4. Crear el formulario HTML

Abrimos:

```
src/main/webapp/index.html
```

Colocamos el siguiente código:

```
</> HTML
<!DOCTYPE html>
<html lang="es">
```



```
<head>
  <meta charset="UTF-8">

  <meta
    name="viewport"
    content="width=device-width, initial-scale=1.0"
  >

  <title>Formulario POST</title>

  <style>
    * {
      box-sizing: border-box;
    }
  </style>
</head>
```

La parte principal del formulario es:

```
HTML
<form action="saludar" method="POST">
```

El atributo:

```
HTML
method="POST"
```

indica que los datos serán enviados dentro del cuerpo de la petición HTTP.

El atributo:

```
HTML
action="saludar"
```

indica que la solicitud será enviada al Servlet asociado con la dirección `/saludar`.

5. Crear el paquete Java

Dentro de:

```
src/main/java
```

creamos la siguiente estructura de carpetas:

```
com/educacionIT/servlet
```

La ruta completa será:

```
src/main/java/com/educacionIT/servlet
```

Dentro creamos el archivo:

```
SaludoServlet.java
```

Es importante respetar las mayúsculas y minúsculas.

6. Crear el Servlet

Código completo de `SaludoServlet.java`:

```
</> Java

package com.educacionIT.servlet;

import java.io.IOException;
import java.io.PrintWriter;

import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

/**
 * Servlet encargado de recibir Los datos enviados
 * por el formulario mediante una petición HTTP POST.
 */
@WebServlet("/saludar")
public class SaludoServlet extends HttpServlet {

    private static final long serialVersionUID = 1L;

    /**
     * Atiende Las solicitudes HTTP POST realizadas
     * a La dirección /saludar.
     *
     * @param request contiene Los datos enviados por el navegador
     * @param response permite construir La respuesta HTTP
     */
    @Override
    protected void doPost(
        HttpServletRequest request,
        HttpServletResponse response
    ) throws ServletException, IOException {

        /*
         * Configuramos UTF-8 antes de Leer Los parámetros.
         */
    }
}
```

```
*/
request.setCharacterEncoding("UTF-8");

/*
 * Leemos los datos enviados por el formulario.
 */
String nombre = request.getParameter("nombre");
String apellido = request.getParameter("apellido");

/*
 * Validamos los datos recibidos.
 */
if (nombre == null || nombre.isBlank()
    || apellido == null || apellido.isBlank()) {

    response.sendError(
        HttpServletResponse.SC_BAD_REQUEST,
        "El nombre y el apellido son obligatorios."
    );

    return;
}

/*
 * Configuramos el tipo de respuesta.
 */
response.setContentType("text/html;charset=UTF-8");

/*
 * Obtenemos el objeto que permite escribir
 * la respuesta HTML.
 */
PrintWriter out = response.getWriter();

out.println("""
    <!DOCTYPE html>
    <html lang="es">

    <head>
        <meta charset="UTF-8">

        <meta
            name="viewport"
            content="width=device-width, initial-scale=1.0"
        >

        <title>Respuesta del servidor</title>

        <style>
            body {
                margin: 0;
                min-height: 100vh;
                display: flex;
            }
        </style>
    """);
```

```

        justify-content: center;
        align-items: center;
        font-family: Arial, sans-serif;
        background-color: #eef2f7;
    }

    .respuesta {
        width: 460px;
        padding: 30px;
        text-align: center;
        background-color: white;
        border-radius: 10px;
        box-shadow: 0 5px 18px rgba(0, 0, 0, 0.15);
    }

    h1 {
        color: #1d3557;
    }

    .datos {
        margin: 20px 0;
        padding: 15px;
        background-color: #f5f7fa;
        border-radius: 6px;
    }

    a {
        display: inline-block;
        padding: 10px 18px;
        color: white;
        text-decoration: none;
        background-color: #1d3557;
        border-radius: 5px;
    }

    a:hover {
        background-color: #457b9d;
    }
</style>
</head>

<body>

    <main class="respuesta">

        <h1>Datos recibidos correctamente</h1>
        """);

    out.println("<div class=\"datos\">");

    out.println(
        "<p><strong>Nombre:</strong> "
        + escaparHtml(nombre)

```

```

        + "</p>"
    );

    out.println(
        "<p><strong>Apellido:</strong> "
        + escaparHtml(apellido)
        + "</p>"
    );

    out.println("</div>");

    out.println(
        "<p>Hola, <strong>"
        + escaparHtml(nombre)
        + " "
        + escaparHtml(apellido)
        + "</strong>.</p>"
    );

    out.println("""
        <p>
            Esta página fue generada dinámicamente
            por un Servlet Java.
        </p>

        <a href="index.html">
            Volver al formulario
        </a>

        </main>

    </body>

    </html>
    """);
}

/**
 * Reemplaza caracteres especiales para evitar
 * que sean interpretados como código HTML.
 *
 * @param texto texto ingresado por el usuario
 * @return texto seguro para mostrar en una página HTML
 */
private String escaparHtml(String texto) {

    if (texto == null) {
        return "";
    }

    return texto
        .replace("&", "&amp;")
        .replace("<", "&lt;");
}

```

```
.replace(">", "&gt;")  
.replace("\"", "&quot;")  
.replace("'", "&#39;");  
  
}  
}
```

7. Relación entre el formulario y el Servlet

En el formulario tenemos:

 HTML

```
<form action="saludar" method="POST">
```

En el Servlet tenemos:

 Java

```
@WebServlet("/saludar")
```

Ambos valores deben coincidir:

Formulario		Servlet
action="saludar"	—————>	@WebServlet("/saludar")

Cuando el usuario presiona el botón, Tomcat busca el Servlet que atiende la URL

/saludar .

8. Relación entre los campos y `getParameter()`

En HTML:

 HTML

```
<input  
  type="text"  
  name="nombre"  
>
```

En Java:

 Java

```
String nombre = request.getParameter("nombre");
```

En HTML:

HTML

```
<input
  type="text"
  name="apellido"
>
```

En Java:

Java

```
String apellido = request.getParameter("apellido");
```

La relación es:

Formulario HTML**Servlet Java**

```
name="nombre"
```

```
getParameter("nombre"
)
```

```
name="apellido"
```

```
getParameter("apellid
o")
```

El atributo importante para enviar los datos es `name`, no `id`.

9. Configurar `web.xml`

Abrimos:

```
src/main/webapp/WEB-INF/web.xml
```

Colocamos:

XML

```
<?xml version="1.0" encoding="UTF-8"?>

<web-app
  xmlns="https://jakarta.ee/xml/ns/jakartaee"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="
    https://jakarta.ee/xml/ns/jakartaee
    https://jakarta.ee/xml/ns/jakartaee/web-app_6_0.xsd"
  version="6.0">

  <display-name>
    Formulario POST EducaciónIT
  </display-name>
```

```

    <welcome-file-list>
      <welcome-file>index.html</welcome-file>
    </welcome-file-list>

  </web-app>

```

No es necesario declarar el Servlet dentro de `web.xml` porque utilizamos la anotación:

```

</> Java
@WebServlet("/saludar")

```

10. Verificar la estructura

Antes de compilar, la estructura debe quedar así:

```

formulario-post
|
├── pom.xml
|
└── src
    ├── main
    │   ├── java
    │   │   ├── com
    │   │   │   ├── educacionIT
    │   │   │   │   ├── servlet
    │   │   │   │   │   └── SaludoServlet.java
    │   │   └── webapp
    │   │       ├── index.html
    │   │       ├── WEB-INF
    │   │       │   └── web.xml
    └── test

```

El encabezado del Servlet debe coincidir con la ubicación de carpetas:

```

</> Java
package com.educacionIT.servlet;

```

La correspondencia es:

```

src/main/java/com/educacionIT/servlet
      ↓
package com.educacionIT.servlet;

```

11. Compilar el proyecto

Desde la carpeta raíz, donde está el `pom.xml` , ejecutamos:

```
</> Bash
```

```
mvn clean package
```

Si todo está correcto, Maven mostrará:

```
BUILD SUCCESS
```

Dentro de la carpeta `target` se generará:

```
target/formulario-post.war
```

12. Desplegar la aplicación en Tomcat

Suponiendo que Tomcat se encuentra instalado en:

```
C:\apache-tomcat-10.1
```

copiamos:

```
target/formulario-post.war
```

dentro de:

```
C:\apache-tomcat-10.1\webapps
```

La ubicación final será:

```
C:\apache-tomcat-10.1\webapps\formulario-post.war
```

13. Iniciar Tomcat

Desde una terminal:

```
</> Bash
```

```
cd C:\apache-tomcat-10.1\bin
```

Para iniciar Tomcat:

```
</> Bash
```

```
startup.bat
```

Durante el desarrollo es más conveniente ejecutar:

```
</> Bash
```

```
catalina.bat run
```

Este comando mantiene la consola abierta y permite observar errores de compilación, despliegue o ejecución.

14. Abrir la aplicación

Abrimos el navegador y escribimos:

```
http://localhost:8080/formulario-post/
```

Se mostrará el formulario.

Ingresamos, por ejemplo:

```
Nombre: Ana
```

```
Apellido: López
```

Presionamos:

```
Enviar mediante POST
```

El navegador enviará la petición a:

```
http://localhost:8080/formulario-post/saludar
```

El Servlet generará la respuesta:

```
Datos recibidos correctamente
```

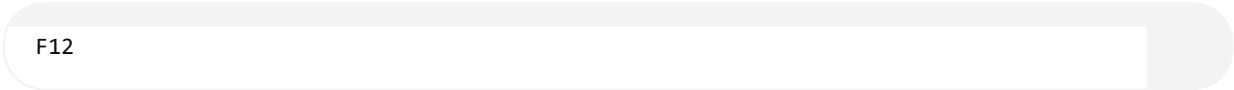
```
Nombre: Ana
```

```
Apellido: López
```

```
Hola, Ana López.
```

15. Comprobar que se utilizó POST

Abrimos las herramientas de desarrollo del navegador:

F12

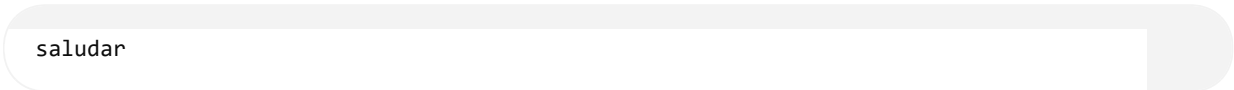
Ingresamos en la pestaña:

Network

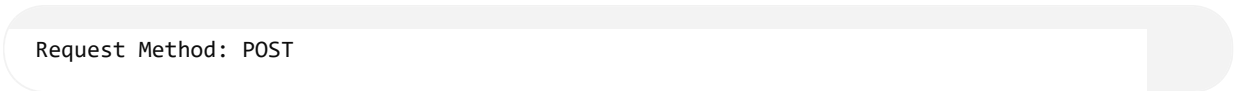
O:

Red

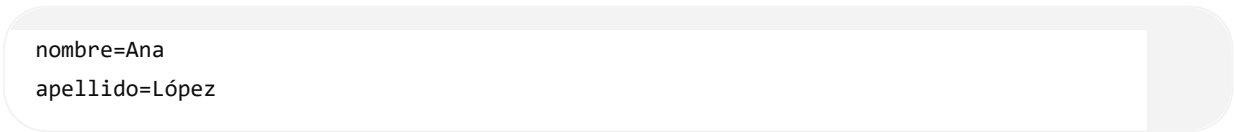
Enviamos nuevamente el formulario y seleccionamos la solicitud:

saludar

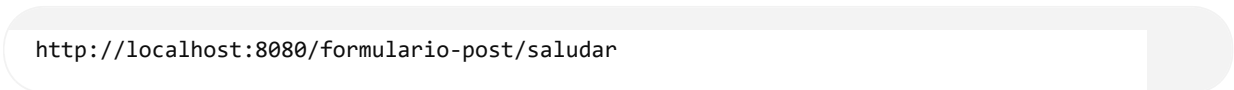
Podremos observar:

Request Method: POST

En la sección de datos del formulario aparecerá:

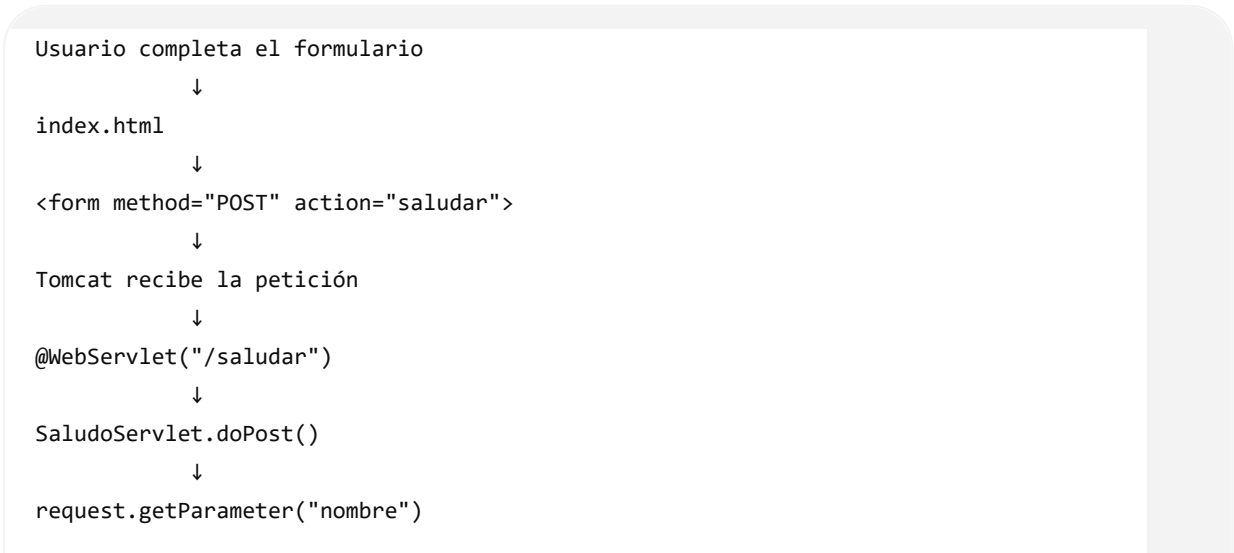
nombre=Ana
apellido=López

La URL será:

http://localhost:8080/formulario-post/saludar

Los datos no aparecen en la URL porque fueron enviados dentro del cuerpo de la petición.

16. Flujo completo de la aplicación



```
graph TD; A[Usuario completa el formulario] --> B[index.html]; B --> C["<form method='POST' action='saludar'>"]; C --> D[Tomcat recibe la petición]; D --> E["@WebServlet(\"/saludar\")"]; E --> F[SaludoServlet.doPost()]; F --> G[request.getParameter("nombre")];
```

Usuario completa el formulario
↓
index.html
↓
<form method="POST" action="saludar">
↓
Tomcat recibe la petición
↓
@WebServlet("/saludar")
↓
SaludoServlet.doPost()
↓
request.getParameter("nombre")

```
request.getParameter("apellido")
    ↓
El Servlet genera HTML
    ↓
El navegador muestra la respuesta
```

17. Métodos principales utilizados

doPost()

```
</> Java
@Override
protected void doPost(
    HttpServletRequest request,
    HttpServletResponse response
) throws ServletException, IOException {
}
```

Se ejecuta cuando el Servlet recibe una petición HTTP POST.

request.getParameter()

```
</> Java
String nombre = request.getParameter("nombre");
```

Permite recuperar un dato enviado por el formulario.

response.setContentType()

```
</> Java
response.setContentType("text/html;charset=UTF-8");
```

Indica que el servidor responderá con contenido HTML codificado en UTF-8.

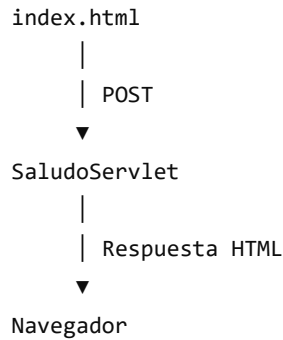
response.getWriter()

```
</> Java
PrintWriter out = response.getWriter();
```

Permite escribir el contenido que será enviado como respuesta al navegador.

18. Resultado conceptual

La aplicación funciona de la siguiente manera:



En esta versión, los identificadores principales quedan actualizados para EducaciónIT:

GroupId Maven:
com.educacionIT

Paquete Java:
com.educacionIT.servlet

```
</> Java
package com.educacionIT.servlet;
```



poner directamente en el POM la ubicación del SERVLET sin usar web.xml y actualizame los pasos

Hay una corrección importante: el `pom.xml` no define la URL o ubicación web del Servlet. El POM administra dependencias, compilación y empaquetado. La ruta del Servlet se configura directamente en Java con:

```
</> Java
@WebServlet("/saludar")
```

Por lo tanto, podemos eliminar completamente `web.xml` y trabajar con:

Formulario HTML → @WebServlet → doPost()

Formulario POST con Java, sin `web.xml`

1. Crear el proyecto Maven

Desde CMD:

```
</> bat
```

```
mvn archetype:generate ^  
-DgroupId=com.educacionIT ^  
-DartifactId=formulario-post ^  
-DarchetypeArtifactId=maven-archetype-webapp ^  
-DinteractiveMode=false
```

En una sola línea:

```
</> Bash  
mvn archetype:generate -DgroupId=com.educacionIT -DartifactId=formulario-post -DarchetypeArt:
```

Ingresamos al proyecto:

```
</> Bash  
cd formulario-post
```

Lo abrimos en Visual Studio Code:

```
</> Bash  
code .
```

2. Estructura final

La aplicación quedará así:

```
formulario-post  
|  
├─ pom.xml  
|  
└─ src  
    └─ main  
        ├── java  
        │   ├── com  
        │   │   └─ educacionIT  
        │   │       └─ servlet  
        │   │           └─ SaludoServlet.java  
        └─ webapp  
            └─ index.html
```

En esta versión eliminamos:

```
src/main/webapp/WEB-INF/web.xml
```

No será necesario porque el Servlet se registrará mediante la anotación `@WebServlet`.

3. Configurar pom.xml

Reemplazamos completamente el archivo pom.xml por:

</> XML

```
<?xml version="1.0" encoding="UTF-8"?>

<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="
    http://maven.apache.org/POM/4.0.0
    https://maven.apache.org/xsd/maven-4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>

  <groupId>com.educacionIT</groupId>
  <artifactId>formulario-post</artifactId>
  <version>1.0-SNAPSHOT</version>

  <!-- El proyecto será una aplicación web -->
  <packaging>war</packaging>

  <properties>
    <maven.compiler.release>17</maven.compiler.release>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>

  <dependencies>

    <!--
      API necesaria para crear Servlets.

      Se utiliza Jakarta porque la aplicación será
      desplegada en Tomcat 10.
    -->
    <dependency>
      <groupId>jakarta.servlet</groupId>
      <artifactId>jakarta.servlet-api</artifactId>
      <version>6.0.0</version>
      <scope>provided</scope>
    </dependency>

  </dependencies>

  <build>

    <!--
      Nombre del archivo WAR y del contexto
      que tendrá la aplicación en Tomcat.
    -->
    <finalName>formulario-post</finalName>
```

```

<!--
    Ubicación de Las clases Java.
    Esta es La ubicación estándar de Maven.
-->
<sourceDirectory>src/main/java</sourceDirectory>

<plugins>

    <!-- Compila el proyecto con Java 17 -->
    <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-compiler-plugin</artifactId>
        <version>3.13.0</version>

        <configuration>
            <release>17</release>
        </configuration>
    </plugin>

    <!-- Genera el archivo WAR -->
    <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-war-plugin</artifactId>
        <version>3.4.0</version>

        <configuration>

            <!-- Permite construir el WAR sin web.xml -->
            <failOnMissingWebXml>false</failOnMissingWebXml>

            <!-- Ubicación de Los archivos HTML y recursos web -->
            <warSourceDirectory>
                src/main/webapp
            </warSourceDirectory>

        </configuration>
    </plugin>

</plugins>

</build>

</project>

```

¿Qué ubicaciones define el POM?

El POM puede indicar dónde están las clases Java:

</> XML

```
<sourceDirectory>src/main/java</sourceDirectory>
```


Y dónde están los archivos web:

</> XML

```
<warSourceDirectory>
  src/main/webapp
</warSourceDirectory>
```

Pero la ruta HTTP del Servlet no se configura en el POM. Se establece en la propia clase Java:

</> Java

```
@WebServlet("/saludar")
```

La separación queda así:

Configuración	Lugar
Ubicación de clases Java	<code>pom.xml</code>
Ubicación de archivos HTML	<code>pom.xml</code>
Dependencia Servlet	<code>pom.xml</code>
Generación del WAR	<code>pom.xml</code>
Ruta HTTP del Servlet	<code>@WebServlet</code>
Procesamiento de POST	<code>doPost()</code>

4. Crear el formulario

Creamos o modificamos:

```
src/main/webapp/index.html
```

Código completo:

HTML

```
<!DOCTYPE html>
<html lang="es">

<head>
```

```
<meta charset="UTF-8">

<meta
  name="viewport"
  content="width=device-width, initial-scale=1.0"
>

<title>Formulario POST</title>

<style>
  * {
    box-sizing: border-box;
  }
```

La parte fundamental es:

HTML

```
<form action="saludar" method="POST">
```

Esto indica que el navegador enviará los datos mediante POST a la ruta:

saludar

5. Crear el paquete del Servlet

Creamos manualmente la ruta:

src/main/java/com/educacionIT/servlet

Dentro creamos:

SaludoServlet.java

La ubicación queda:

src/main/java/com/educacionIT/servlet/SaludoServlet.java

6. Crear SaludoServlet.java

Java

```
package com.educacionIT.servlet;

import java.io.IOException;
```

```
import java.io.PrintWriter;

import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

/**
 * Servlet encargado de recibir datos mediante POST
 * y generar una respuesta HTML.
 */
@WebServlet("/saludar")
public class SaludoServlet extends HttpServlet {

    private static final long serialVersionUID = 1L;

    /**
     * Atiende las solicitudes HTTP POST enviadas
     * a la ruta /saludar.
     *
     * @param request solicitud enviada por el navegador
     * @param response respuesta generada por el servidor
     */
    @Override
    protected void doPost(
        HttpServletRequest request,
        HttpServletResponse response
    ) throws ServletException, IOException {

        /**
         * Configuramos la codificación antes
         * de recuperar los parámetros.
         */
        request.setCharacterEncoding("UTF-8");

        /**
         * Recuperamos los datos enviados por el formulario.
         * Los nombres deben coincidir con los atributos name.
         */
        String nombre = request.getParameter("nombre");
        String apellido = request.getParameter("apellido");

        /**
         * Validación realizada del lado del servidor.
         */
        if (nombre == null || nombre.isBlank()
            || apellido == null || apellido.isBlank()) {

            response.sendError(
                HttpServletResponse.SC_BAD_REQUEST,
                "El nombre y el apellido son obligatorios."
            );
        }
    }
}
```

```
        return;
    }

    /*
     * Indicamos que la respuesta será HTML
     * codificado en UTF-8.
     */
    response.setContentType("text/html;charset=UTF-8");

    PrintWriter out = response.getWriter();

    out.println("""
        <!DOCTYPE html>
        <html lang="es">

        <head>
            <meta charset="UTF-8">

            <meta
                name="viewport"
                content="width=device-width, initial-scale=1.0"
            >

            <title>Respuesta del Servlet</title>

            <style>
                body {
                    min-height: 100vh;
                    margin: 0;
                    display: flex;
                    justify-content: center;
                    align-items: center;
                    font-family: Arial, sans-serif;
                    background-color: #eef2f7;
                }

                .respuesta {
                    width: 460px;
                    padding: 30px;
                    text-align: center;
                    background-color: white;
                    border-radius: 10px;
                    box-shadow: 0 5px 18px rgba(0, 0, 0, 0.15);
                }

                h1 {
                    color: #1d3557;
                }

                .datos {
                    margin: 20px 0;
                    padding: 15px;
                }
            </style>
        """);
}
```

```

        background-color: #f5f7fa;
        border-radius: 6px;
    }

    a {
        display: inline-block;
        margin-top: 15px;
        padding: 10px 18px;
        color: white;
        text-decoration: none;
        background-color: #1d3557;
        border-radius: 5px;
    }

    a:hover {
        background-color: #457b9d;
    }
</style>
</head>

<body>

    <main class="respuesta">

        <h1>Datos recibidos correctamente</h1>
    """);

out.println("<div class=\"datos\">");

out.println(
    "<p><strong>Nombre:</strong> "
    + escaparHtml(nombre)
    + "</p>"
);

out.println(
    "<p><strong>Apellido:</strong> "
    + escaparHtml(apellido)
    + "</p>"
);

out.println("</div>");

out.println(
    "<p>Hola, <strong>"
    + escaparHtml(nombre)
    + " "
    + escaparHtml(apellido)
    + "</strong>.</p>"
);

out.println("""
    <p>

```

```

        Esta respuesta fue generada
        por un Servlet Java.
    </p>

    <a href="index.html">
        Volver al formulario
    </a>

</main>

</body>

</html>
""");
}

/**
 * Escapa caracteres que podrían ser interpretados
 * como código HTML.
 *
 * @param texto texto ingresado por el usuario
 * @return texto seguro para mostrar
 */
private String escaparHtml(String texto) {

    if (texto == null) {
        return "";
    }

    return texto
        .replace("&", "&amp;")
        .replace("<", "&lt;")
        .replace(">", "&gt;")
        .replace("\\\"", "&quot;")
        .replace("'", "&#39;");
}
}

```

7. Registrar el Servlet sin `web.xml`

La instrucción que registra el Servlet es:

```

</> Java
@WebServlet("/saludar")

```

Esto le indica a Tomcat:

Cuando llegue una petición a /saludar,

```
ejecutar SaludoServlet.
```

No necesitamos escribir en `web.xml` :

```
</> XML
<servlet>
  ...
</servlet>
```

ni:

```
</> XML
<servlet-mapping>
  ...
</servlet-mapping>
```

La anotación reemplaza ambas configuraciones.

8. Relación entre HTML y Servlet

En el formulario:

```
</> HTML
<form action="saludar" method="POST">
```

En la clase Java:

```
</> Java
@WebServlet("/saludar")
```

La relación es:

```
action="saludar"
  |
  ▼
@WebServlet("/saludar")
```

Cuando el usuario envía el formulario:

```
POST /formulario-post/saludar
```

Tomcat ejecuta:

```
</> Java
```

```
SaludoServlet.doPost()
```

9. Relación entre los campos y Java

En HTML:

HTML

```
<input name="nombre">
```

En Java:

Java

```
String nombre = request.getParameter("nombre");
```

En HTML:

HTML

```
<input name="apellido">
```

En Java:

Java

```
String apellido = request.getParameter("apellido");
```

Los nombres tienen que coincidir exactamente:

HTML

Java

```
name="nombre"
```

```
getParameter("nombre")  
)
```

```
name="apellido"
```

```
getParameter("apellid  
o")
```

10. Eliminar web.xml

Podemos borrar completamente:

```
src/main/webapp/WEB-INF/web.xml
```

También podemos eliminar la carpeta `WEB-INF` si queda vacía.

La estructura definitiva será:

```
formulario-post
├── pom.xml
├── src
│   ├── main
│   │   ├── java
│   │   │   ├── com
│   │   │   │   ├── educacionIT
│   │   │   │   │   ├── servlet
│   │   │   │   │   │   └── SaludoServlet.java
│   │   └── webapp
│   │       └── index.html
```

11. Compilar el proyecto

Desde la carpeta que contiene el `pom.xml` :

Bash

```
mvn clean package
```

Si todo está correcto veremos:

BUILD SUCCESS

Maven generará:

target/formulario-post.war

La opción:

XML

```
<failOnMissingWebXml>false</failOnMissingWebXml>
```

permite generar el WAR aunque no exista `web.xml` .

12. Desplegar en Tomcat

Copiamos:

```
target/formulario-post.war
```

en:

```
C:\apache-tomcat-10.1\webapps
```

La ubicación final será:

```
C:\apache-tomcat-10.1\webapps\formulario-post.war
```

13. Iniciar Tomcat

Desde una terminal:

```
</> bat  
cd C:\apache-tomcat-10.1\bin
```

Ejecutamos:

```
</> bat  
catalina.bat run
```

Esta opción es útil porque deja visibles los mensajes del servidor.

También se puede utilizar:

```
</> bat  
startup.bat
```

14. Abrir la aplicación

En el navegador:

```
http://localhost:8080/formulario-post/
```

Tomcat busca automáticamente un archivo inicial como:

```
index.html
```

No necesitamos configurar el `welcome-file` porque `index.html` es uno de los archivos de bienvenida convencionales que el contenedor reconoce.

15. Probar el formulario

Ingresamos, por ejemplo:

Nombre: Ana
Apellido: López

Presionamos:

Enviar mediante POST

La solicitud será enviada a:

`http://localhost:8080/formulario-post/saludar`

Tomcat encontrará:

```
</> Java
@WebServlet("/saludar")
```

y ejecutará:

```
</> Java
doPost()
```

16. Verificar la petición POST

Abrimos las herramientas del navegador con:

F12

Ingresamos en:

Network

Enviamos nuevamente el formulario y seleccionamos la petición:

saludar

Deberíamos observar:

Request Method: POST

En los datos del formulario aparecerá algo equivalente a:

nombre=Ana
apellido=López

La URL no muestra los parámetros:

http://localhost:8080/formulario-post/saludar

porque los datos viajan en el cuerpo de la petición.

17. Flujo final sin `web.xml`

```
Usuario completa el formulario
    ↓
index.html
    ↓
<form method="POST" action="saludar">
    ↓
Tomcat recibe POST /saludar
    ↓
@WebServlet("/saludar")
    ↓
SaludoServlet.doPost()
    ↓
request.getParameter(...)
    ↓
Respuesta HTML
    ↓
Navegador
```

La configuración queda dividida correctamente:

`pom.xml`

- Dependencia Jakarta Servlet
- Versión de Java
- Ubicación de las clases
- Ubicación de los recursos web
- Generación del archivo WAR

`SaludoServlet.java`

- Ruta del Servlet con `@WebServlet`
- Recepción del POST

- └─ Lectura de parámetros
- └─ Generación de la respuesta

La línea que verdaderamente reemplaza la configuración de `web.xml` es:

`</>` Java

```
@WebServlet("/saludar")
```

